UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

University of Canterbury

FINAL EXAM 2025

Prescription Number(s):	**ENCE260-25S2**
Paper Title:	**Computer Systems**

Time allowed: **Two Hours**

Number of pages: **35**

Calculators **ARE** permitted.

ALL Questions **ARE COMPULSORY**.
(There are 4 questions in total.
The maximum total score is 100.)

Family name (in block letters): .....
Given name: .....
Student number: .....

INSTRUCTIONS

You are required to complete this examination by writing your answers into this booklet. Show your working as you complete this examination booklet, so that you can more easily receive partial credit.

If you need more space for your answers, several extra pages can be found at the end of this booklet. Make a note indicating when your work is continued on an extra page.

This page is intentionally blank

Exam starts on Page 4

Q1. Computer Architecture

(a) [1 mark]

How is every piece of information within computers stored?

(b) Logic functions and design:

A	B	C	V
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(i) [2 marks]

From the above truth table, write a Boolean expression for V in "sum-of-minterms" form (i.e., each term in the expression should specify exactly one true state of V).

(ii) [2 marks]

Determine the simplest possible form of the "sum-of-minterms" expression for V. You may use Boolean algebra or a Karnaugh map. You must show all working for full marks:

(iii) [3 marks]

Determine an expression for V using **ONLY** NAND operations.

(iv) [1 mark]

Sketch an implementation of the Boolean function V, using only NAND logic gates.

(c) You are designing a CPU, and you know that the subcomponents shown in Figure 1 are different places information is stored or manipulated.

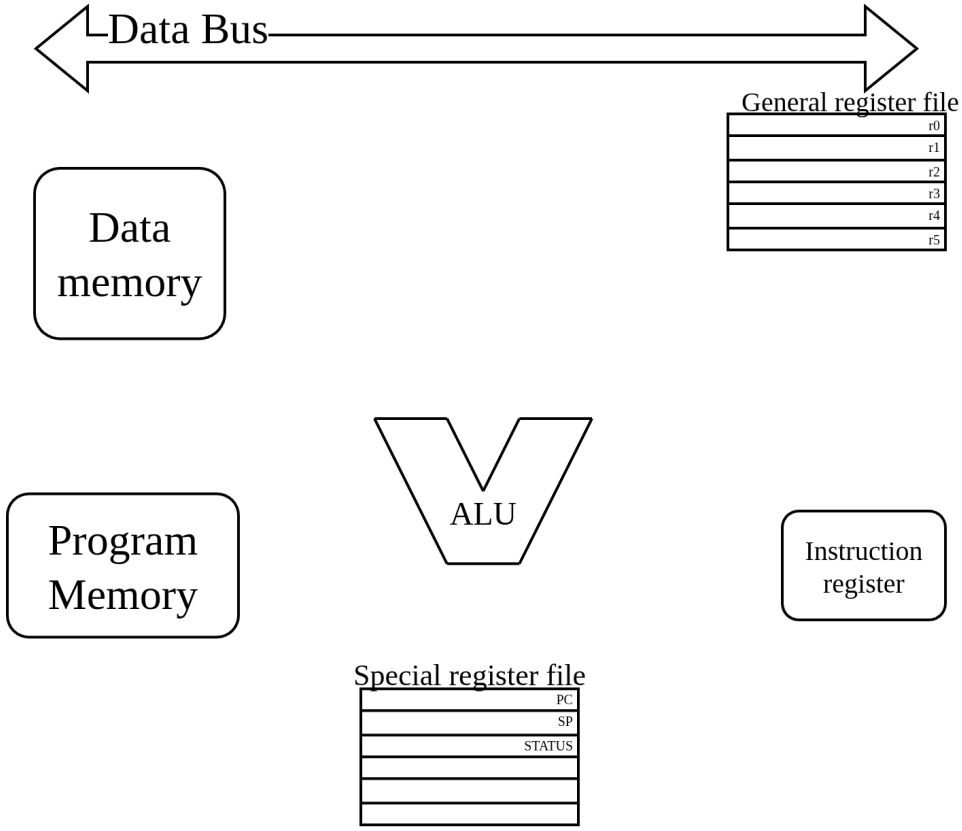


Figure 1: Diagram of various components within an embedded computer.

(i) [4 marks]

Draw **clearly labelled arrows** on Figure 1, showing:

1. The path an instruction takes during a 'fetch' cycle.
2. The path data takes during a 'load' instuction (for example `ldd`).
3. The path(s) data takes during an addition instruction, including flags.

The ATMega32u2 is a good example to use to learn about computer architecture. Its block diagram is shown in Figure 2.

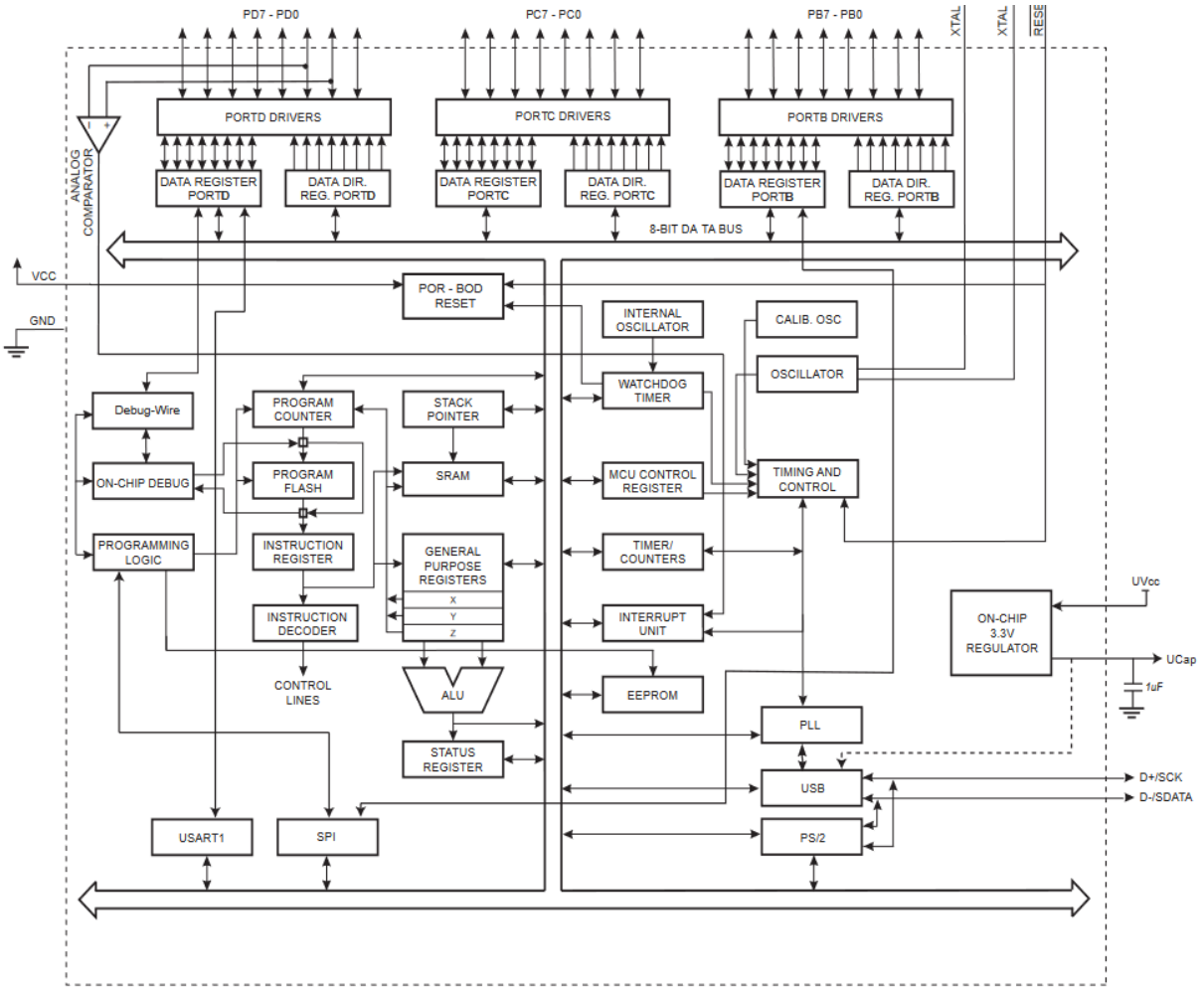


Figure 2: Block diagram of the ATMega32u2.

(ii) [4 marks]

On the diagram in Figure 2, label and identify the components A) through F).

A) A peripheral used to generate a PWM waveform.

B) Somewhere an LED controlled by PWM would be connected to.

C) Somewhere a large array of variables would be stored.

D) The means by which data is transferred throughout the microcontroller.

E) The control and execute unit.

F) The pointer to the next instruction to be run.

(d) Table 1 is the truth table for an individual Full Adder circuit which performs addition on 1-bit numbers. The 4-bit Ripple-Carry Adder shown in Figure 3 uses four Full Adder circuits to perform addition on 4-bit binary numbers.

Table 1: Truth table for a Full Adder.

Inputs			Outputs	
A	B	C_{in} (carry in)	S (sum)	C_{out} (carry out)
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

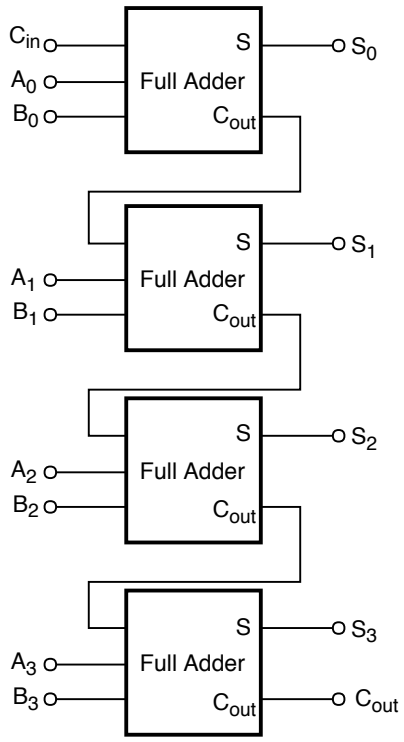


Figure 3: 4-bit Ripple-Carry Adder for context.

Consider now an 8-bit ripple counter, as would be found in an 8-bit ALU.

(i) [1 mark]

The Ripple-Carry Adder is to be used to analyse the base-10 numbers A = 13 and B = 16. Write the 8-bit binary sequences, $A_7A_6A_5A_4A_3A_2A_1A_0$ and $B_7B_6B_5B_4B_3B_2B_1B_0$, that represent the inputs to the Ripple-Carry Adder.

(ii) [2 marks]

You wish to perform the operation $A - B$. State the two required changes necessary to change from a system configured to perform $A + B$ to $A - B$ within an ALU.

(iii) [3 marks]

Give the binary values of the operands an ALU would use when calculating $A - B$, where A and B are as defined in part **(i)**, **AND** calculate the result.

(iv) [2 marks]

Interpret you answer from **(iii)** in two's complement, **AND** give two other interpretations of the binary sequence.

[If you have no answer from above use `0b10100011`.]

Q2. **(a)** Memory is one of the critical parts of any computer device. One of the base units of memory is the flip-flop, for example that shown in Figure 4.

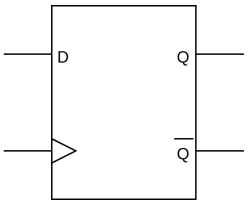


Figure 4: A flip-flop

(i) **[2 marks]**

State two functions that memory must perform, and briefly discuss how a flip-flop achieves these required functions.

(ii) **[1 mark]**

State the difference between sequential logic and combinational logic.

An example of a sequential logic circuit is shown in Figure 5.

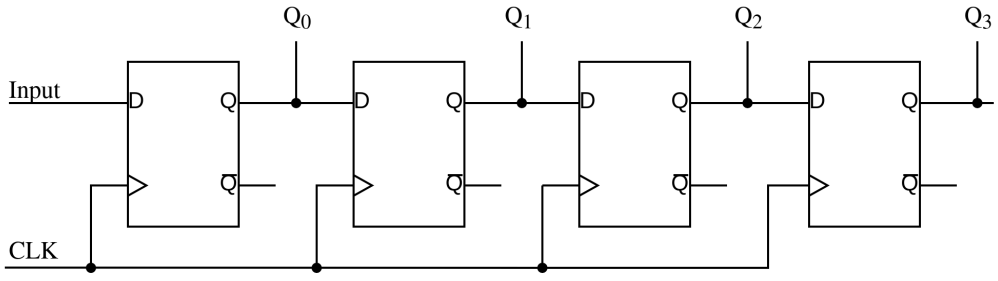


Figure 5: An example 4-bit sequential logic circuit.

(iii) **[1 mark]**

Name the sequential logic circuit shown in Figure 5.

The electrical signals provided to the `input` and the `CLK` connections are shown in Figure 6.

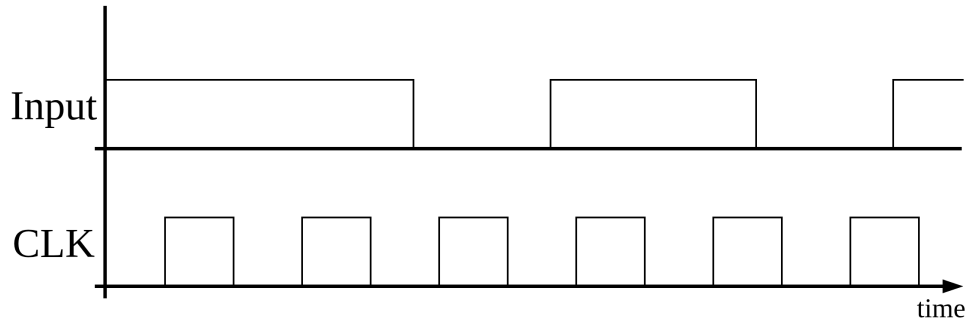
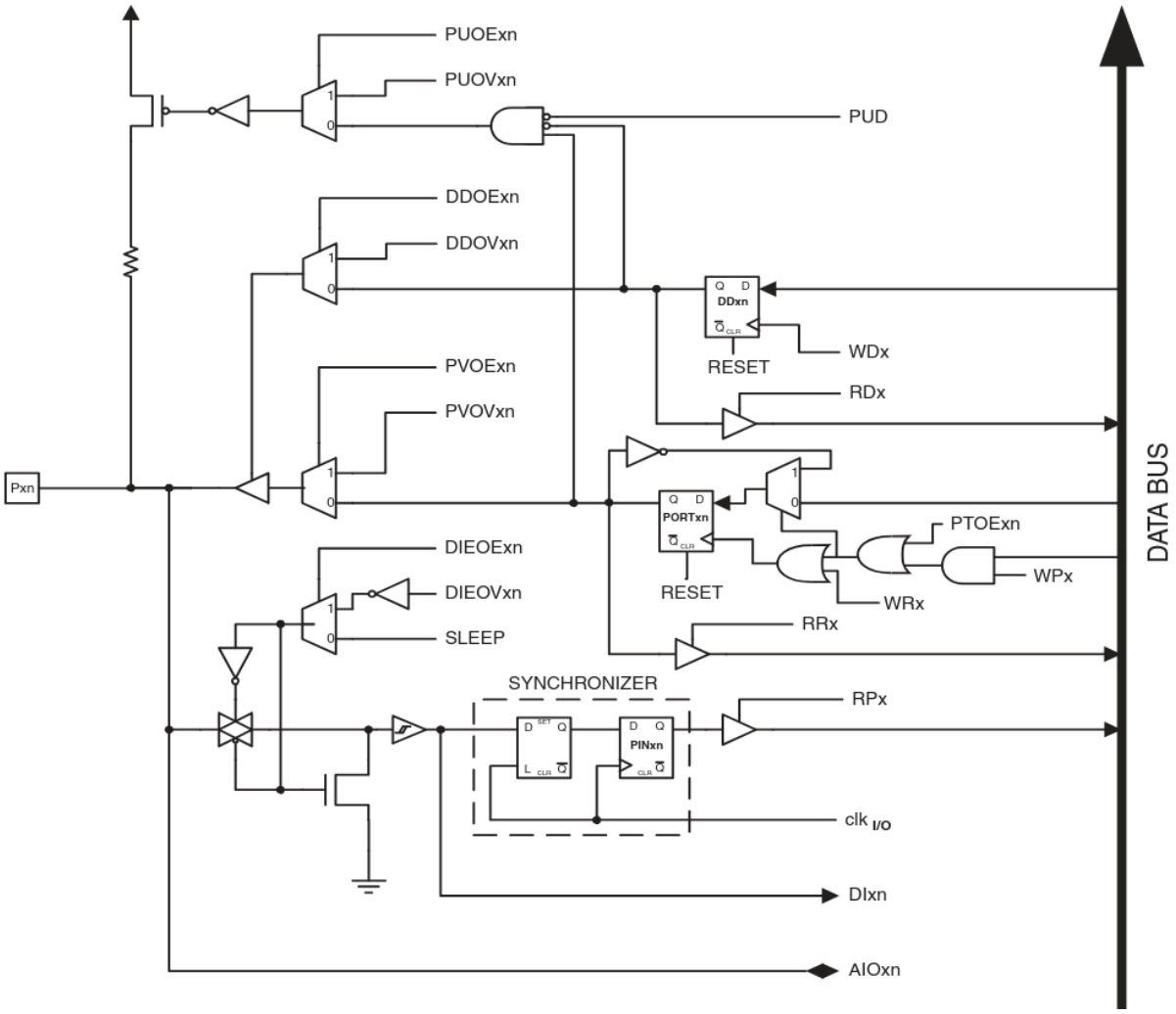


Figure 6: Electrical waveforms provided to the sequential logic circuit.

(iv) [3 marks]

Determine each of the signals $Q_0Q_1Q_2Q_3$ **after the end of the entire waveforms** shown in Figure 6.

(b) Programmable input output functionality



PUOExn:	Pxn PULL-UP OVERRIDE ENABLE
PUOVxn:	Pxn PULL-UP OVERRIDE VALUE
DDOExn:	Pxn DATA DIRECTION OVERRIDE ENABLE
DDOVxn:	Pxn DATA DIRECTION OVERRIDE VALUE
PVOExn:	Pxn PORT VALUE OVERRIDE ENABLE
PVOVxn:	Pxn PORT VALUE OVERRIDE VALUE
DIEOExn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE ENABLE
DIEOVxn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE VALUE
SLEEP:	SLEEP CONTROL
PTOExn:	Pxn, PORT TOGGLE OVERRIDE ENABLE
PUD:	PULLUP DISABLE
WDx:	WRITE DDRx
RDx:	READ DDRx
RRx:	READ PORTx REGISTER
WRx:	WRITE PORTx
RPx:	READ PORTx PIN
WPx:	WRITE PINx
$clk_{I/O}$:	I/O CLOCK
DIxn:	DIGITAL INPUT PIN n ON PORTx
AIOxn:	ANALOG INPUT/OUTPUT PIN n ON PORTx

Figure 7: Diagram of the internal GPIO circuitry of the ATMega32u2, including alternate function circuitry.

Pin PB7 of the ATmega32u2 contains alternate function to act as a PWM output signal. Pin PB7 is currently configured with the bit settings per Table 2:

Table 2: Bit settings for PB7.

Signal	State	Signal	State
PUOEB7	0	PUOVB7	0
PUD	0	DDOEB7	0
DDB7	0	PVOEB7	0
PVOVB7	1	PORTB7	1
(others)			0

(i) [3 marks]

With reference to the function of a multiplexer and a tristate buffer, describe how a GPIO pin such as PB7 is able to act either as a GPIO, or with an alternate function determined by an internal peripheral.

(ii) [2 marks]

With reference to Figure 7, explain in detail how the pin PB7 is programmed to behave according to the bit settings in Table 2.

When considering implementing PWM on pin PB7, you know that you can use the built-in timer/counter peripheral (hardware implementation), or you can simply turn the pin off and on during your main loop with manipulation of the PORTB register (software implementation).

(iii) [3 marks]

State which is the more robust solution, and justify your answer.

This page is intentionally blank.

(c) Table 3: Opcode summaries for AVR instructions on the ATmega32u2

Syntax	Operands	Operation
`sub Rd, Rr`	$0 \leq d \leq 31, 0 \leq r \leq 31$	Rd ← (Rd-Rr)
Control registers	Opcode	Flags
PC ← PC + 1	0001 10rd dddd rrrr	H, S, V, N, Z, C

Syntax	Operands	Operation
`subi Rd, K`	$16 \leq d \leq 31, 0 \leq K \leq 255$	Rd ← (Rd-K)
Control registers	Opcode	Flags
PC ← PC + 1	0101 KKKK dddd KKKK	H, S, V, N, Z, C

Syntax	Operands	Operation
`cp Rd, Rr`	$0 \leq d \leq 31, 0 \leq r \leq 31$	Rd-K
Control registers	Opcode	Flags
PC ← PC + 1	0001 01rd dddd rrrr	H, S, V, N, Z, C

Syntax	Operands	Operation
`breq k`	$-64 \leq k \leq 63$	Is last result = 0?
Control registers	Opcode	Flags
PC ← PC + 1 (if Z = 0) PC ← PC + k + 1 (if Z = 1)	1111 00kk kkkk k001	None

Syntax	Operands	Operation
`brlt k`	$-64 \leq k \leq 63$	Is last result < 0?
Control registers	Opcode	Flags
PC ← PC + 1 (if S = 0) PC ← PC + k + 1 (if S = 1)	1111 00kk kkkk k100	None

(i) [3 marks]

Discuss the differences between the `sub` and `subi` instructions.

Examining your ATmega32u2, you find the following sequences of 1s and 0s when `PC = 0x00f8`.

PC	opcode (binary)
...	
0x00f8:	0001 1011 0010 0101
0x00f9:	1111 0001 0000 0100
...	

Register	Value (hex)
...	
R16	01
R17	00
R18	c0
R19	1a
R20	3c
R21	c2
R22	00
R23	f0
...	

Figure 8: Example of some AVR instructions as compiled for the ATMega32u2.

(ii) [2 marks]

What register(s) are read from and/or written to by the instruction at location `PC = 0x00f8`?

(iii) [3 marks]

What is the value of `PC` immediately after the instructions at locations `0x00f8` and `0x00f9` are carried out?

(iv) [2 marks]

Explain why the `cp` instruction might be used instead of the `sub` instruction.

Q3. Embedded Systems 1

(a) An ATmega32u2 micrcocontroller is connected to four LEDs (L1 to L4) and two pushbutton switches (SW1 and SW2) as shown in Figure 9.

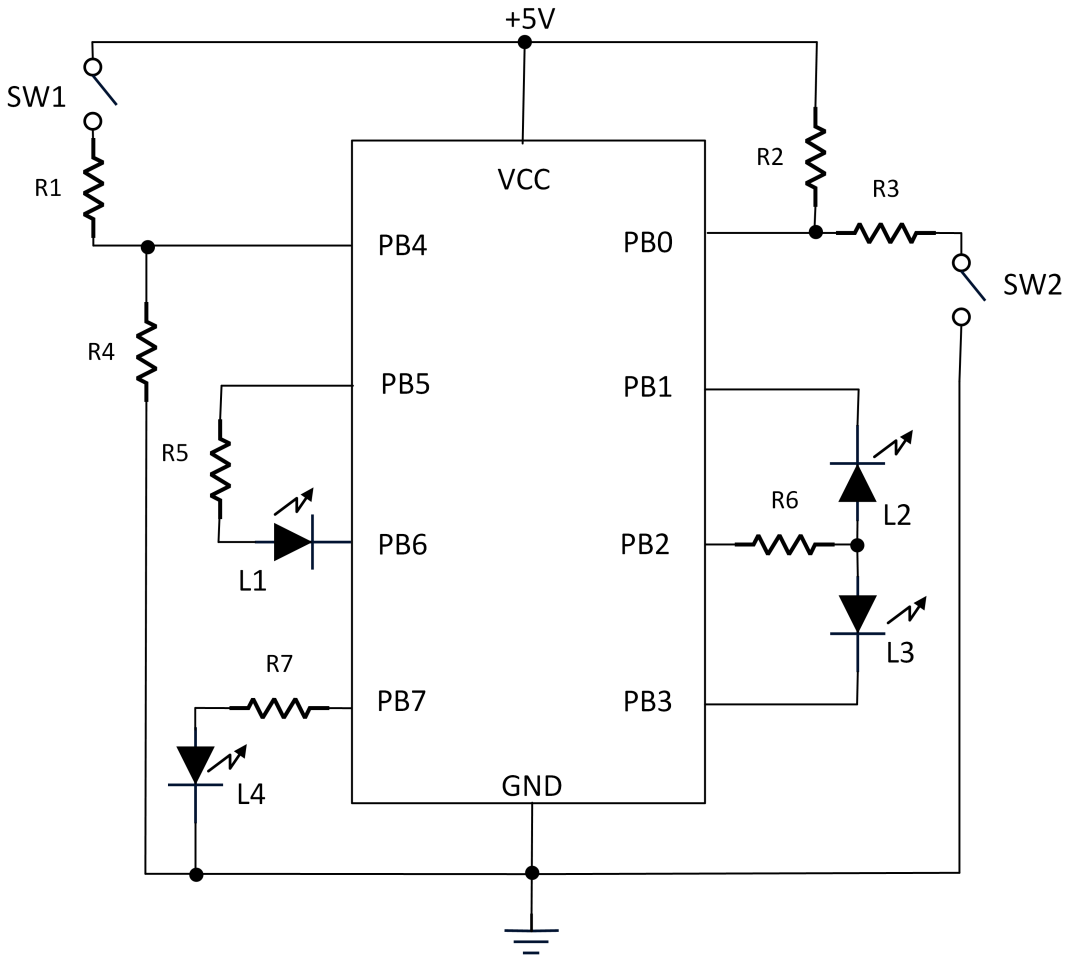


Figure 9: Schematic showing four LEDs and two pushbuttons connected to an ATmega32u2 microcontroller for Q3.

(i) [2 marks]

What is the purpose of resistor R4?

(ii) [2 marks]

What **decimal** value should be stored in the Data Direction Register for Port B (DDRB)?

(iii) [3 marks]

Write an `if` conditional statement in C that evaluates to `True` only when simultaneously the pushbutton SW2 is pushed and pushbutton SW1 is not pushed.

(iv) [2 marks]

If `PORTB = 0xC6` , which of the four LEDs will be on?

(v) [1 mark]

Your colleague has written a macro to set the bit at position X to 1, and placed it in the file `system.h`:

```
#define BIT(X) 1<<(X)
```

The BIT macro is an example of *abstraction*. What is the advantage of *abstraction* in embedded systems programming?

(vi) [2 marks]

Your colleague's macro has a flaw. What is the decimal value of `BIT(3)+1`? Does it equal `0b00001001` ? [Hint: Addition (+) has higher precedence than bit shift (<<) in C.]

(vii) [1 mark]

Rewrite the macro to fix this flaw in the answer box below.

(viii) [2 marks]

Write a C statement using the BIT macro that will toggle L4 without affecting the other outputs.

(ix) [3 marks]

You and your colleague have a shared git remote repository. Assuming your current directory includes `system.h` and that your current directory is in your existing local git repository, in the following boxes write the git command(s) that both you and your colleague need to enter so that the change to the BIT macro is updated in your colleague's local repository.

YOU:

YOUR COLLEAGUE:

Table 4: The ASCII character table for Q3(c).

0	NUL	32		64	@	96	‘
1	^A	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	^D	36	$	68	D	100	d
5	^E	37	%	69	E	101	e
6	^F	38	&	70	F	102	f
7	BELL	39	’	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	z
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	^N	46	.	78	N	110	n
15	^O	47	/	79	O	111	o
16	^P	48	0	80	P	112	p
17	XON	49	1	81	Q	113	q
18	^R	50	2	82	R	114	r
19	XOFF	51	3	83	S	115	s
20	^T	52	4	84	T	116	t
21	^U	53	5	85	U	117	u
22	^V	54	6	86	V	118	v
23	^W	55	7	87	W	119	w
24	^X	56	8	88	X	120	x
25	^Y	57	9	89	Y	121	y
26	^Z	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

(b) [3 marks]

The University of Canterbury Fun Kit Version 4 (UCFK4) can communicate with another UCFK4 via infra-red transmission. Is the UCFK4 an example of *simplex*, *half duplex* or *full duplex* communication. Justify your answer.

(c) [4 marks]

Data is transmitted serially from one UART to another. Assume the UARTs have been configured for 7-bit data, the MSB is transmitted first, the idle state is high, and there are two stop bits, one start bit, and an even parity bit. Sketch the waveform that should be transmitted from the UART to transmit the ASCII character `F` in the answer box below, using the answer lines for any calculations. An ASCII table is shown in Table 4.

Q4. Embedded Systems 2

(a) The `bounce` app on the UCFK4 bounces a ball around the LED matrix display. The makefile for this app is shown in Listing 1.

Listing 1: Makefile for Question 4(a).

```
CC = avr-gcc
CFLAGS = -mmcu=atmega32u2 -Os -Wall
OBJCOPY = avr-objcopy

all: bounce.out

bounce.o: bounce.c system.h display.h pacer.h
	$(CC) -c $(CFLAGS) $< -o $@

pio.o: pio.c pio.h system.h
	$(CC) -c $(CFLAGS) $< -o $@

system.o: system.c system.h
	$(CC) -c $(CFLAGS) $< -o $@

timer.o: timer.c system.h timer.h
	$(CC) -c $(CFLAGS) $< -o $@

display.o: display.c system.h display.h ledmat.h
	$(CC) -c $(CFLAGS) $< -o $@

ledmat.o: ledmat.c pio.h system.h ledmat.h
	$(CC) -c $(CFLAGS) $< -o $@

pacer.o: pacer.c system.h timer.h pacer.h
	$(CC) -c $(CFLAGS) $< -o $@

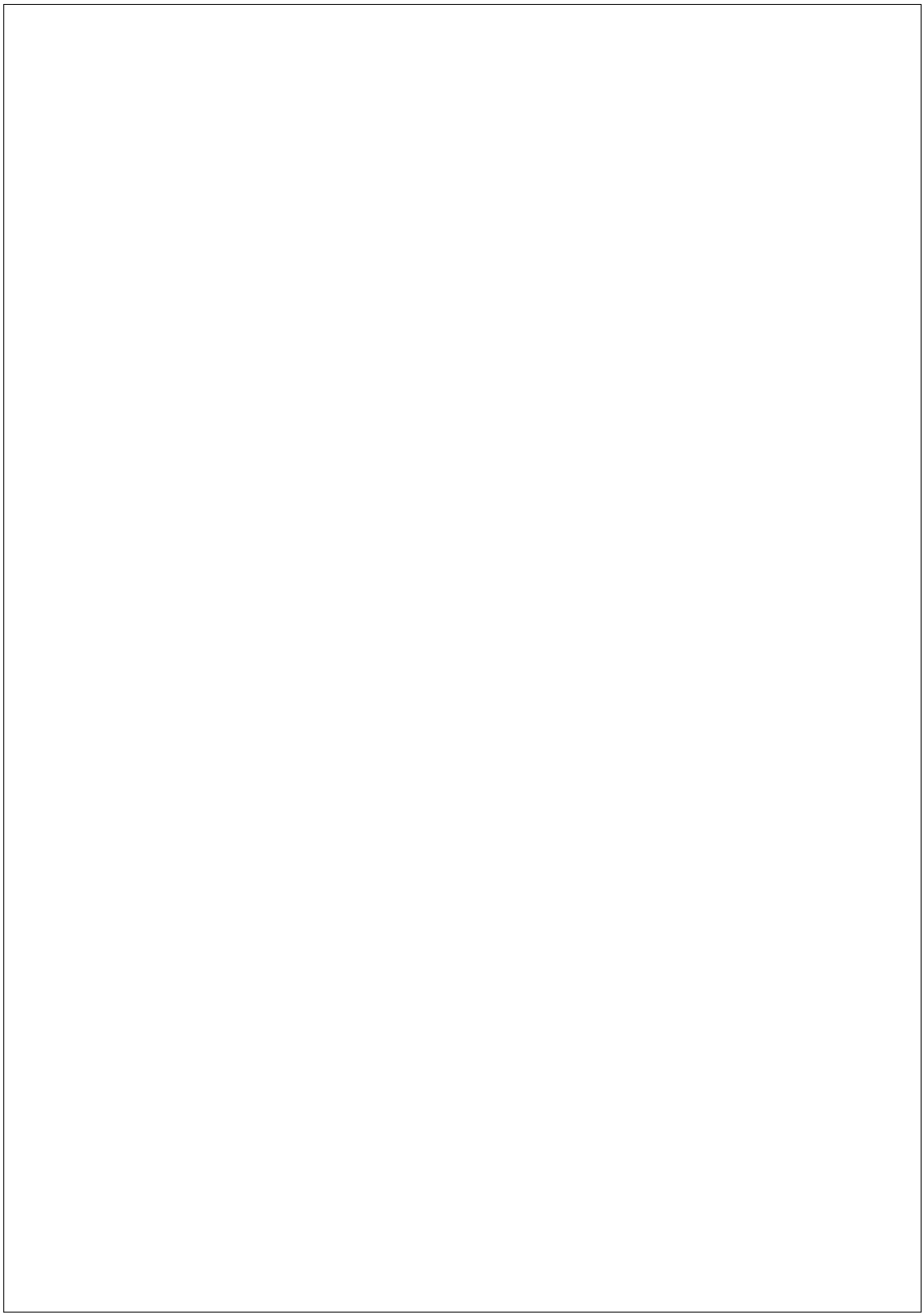
bounce.out: bounce.o pio.o system.o timer.o \
        display.o ledmat.o pacer.o
    $(CC) $(CFLAGS) $^ -o $@ -lm

bounce.hex: bounce.out
    $(OBJCOPY) -O ihex bounce.out bounce.hex

.PHONY: program
program: bounce.hex
	dfu-programmer atmega32u2 erase; dfu-programmer \
	    atmega32u2 flash bounce.hex; \
	        dfu-programmer atmega32u2 start
```

(i) [4 marks]

In the box below, draw the file dependency graph of the files used to build the executable file. For clarity and ease in drawing your graph, you may ignore all dependencies on `system.h`.



(ii) [3 marks]

Of the file types in the `bounce` app directory after the `make program` command is successfully executed, which should be included in a version control system such as git? Justify your answer.

(iii) [2 marks]

The makefile in Listing 1 contains a number of automatic variables as indicated by the $ symbol. Write out Line 30 of Listing 1 in full.

This page is intentionally blank.

(b) The C program in Listing 2 for the ATmega32u2 microcontroller has been written to produce an LED that flashes at 20 Hz. The TCCR0B configuration register is shown in Figure 10, and the TCNT0 register in Figure 11. The LED is in an active high configuration. Given a CPU frequency of 2 MHz, answer the following.

Listing 2: Source listing for Q4(b)

```
#include "system.h"
#include "led.h"

#define CPU_F 2000000          /* Hz */
#define PRESCALER

void pacer (uint8_t ticks) {
   while (TCNT0 < ticks)
      continue;
   TCNT0 = 0;
}

int main (void)
{
    system_init ();
    led_init ();

    TCCR0B =
    TCNT0 = 0;

    uint8_t LED_frequency = 20; /* Hz */
    uint8_t delay_ticks;

    delay_ticks =

    while (1)  {

      led_set (LED1,1);    /* Turn LED On */
      pacer (delay_ticks);
      led_set (LED1,0);    /* Turn LED Off */
      pacer (delay_ticks);
    }

   return 0;

}
```

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 15-9. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/(No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Figure 10: TCCR0B register bits for Q4(b).

TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
0x26 (0x46)	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figure 11: TCNT0 register bits for Q4(b).

(i) [1 mark]

What is the maximum value that TCNT0 can count up to?

(ii) [2 marks]

What is the `pacer` period (in seconds) required in Listing 2 for the LED to flash at 20 Hz?

(iii) [5 marks]

Lines 5 and 18 in Listing 2 have not yet been completed. Use the answer lines below to calculate the value for TCCR0B that can flash the LED at 20 Hz and provide the best temporal resolution without overflow of TCNT0. Complete Lines 5 and 18 in the answer box provided.

(iv) [2 marks]

Complete Line 24 in Listing 2 in the following answer box, to calculate `delay_ticks` in terms of `CPU_F`, `PRESCALER` and `LED_frequency`.

(c) The source and header files for a pacer module are given in Listings 3 and 4 respectively. Use these two files to answer the following.

Listing 3: pacer.h for Q4(c)

```
#ifndef PACER_H
#define PACER_H

#include "system.h"

typedef uint16_t pacer_rate_t;

void pacerWait (void);

void init_pacer (pacer_rate_t pacer_rate);

#endif
```

Listing 4: pacer.c for Q4(c)

```
#include "system.h"
#include "timer.h"
#include "pacer.h"

static timer_tick_t pacer_period_ticks;

void init_pacer (pacer_rate_t pacer_rate)
{
    timer_init ();
    pacer_period_ticks = TIMER_RATE / pacer_rate;
}

void pacerWait (void)
{
    static timer_tick_t when = 0;

    timer_wait_until (when);
    when += pacer_period_ticks;
}
```

(i) [1 mark]

Identify a *public* function or variable in the `pacer` module. If no *public* functions or variables exist, then state this.

(ii) [1 mark]

Identify a *private* function or variable in the `pacer` module. If no *private* functions or variables exist, then state this.

(iii) [1 mark]

What is the purpose of lines 1, 2 and 12 in Listing 3?

(iv) [3 marks]

Identify three (and not more than three) cases of poor programming practice in the `pacer` module.

END OF QUESTIONS

Extra answer space (optional)

Extra answer space (optional)

END OF EXAM